Binary Search Trees

In-Class Work

EECE 7205: Fundamentals of Computer Engineering Fall 2026

MyungJin Suh October 22, 2026

Same array in every problem: $arr = \{50, 30, 70, 20, 40, 60, 80, 35, 45, 65, 75\}$, $n = 11$. A new insert is always a leaf. Height of an empty tree is -1, so one node has height 0. ● added / found / rewritten ○ compared ○ deleted ◌ NULL

Problem 1. Insert

Case 1. Insert 50 into an empty tree

Rules: I1

`root` is NULL, so 50 becomes the root. No comparison.

Before

(empty)

After



Case 2. Insert 30

Rules: I2, I1

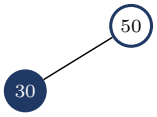
$30 < 50$, and the left child of 50 is NULL. Attach 30 there. 1 comparison.

```
1: if key < cur.key then
2:     if cur.left = NULL then
3:         cur.left ← newNode                ▷ I1
4:         return
5:     end if
6:     cur ← cur.left                        ▷ I2
7: end if
```

Before



After



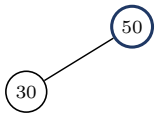
Case 3. Insert 70

Rules: I3, I1

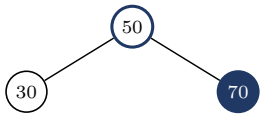
$70 > 50$, and the right child of 50 is NULL. Attach 70 there. 1 comparison.

```
1: if key > cur.key then
2:     if cur.right = NULL then
3:         cur.right ← newNode               ▷ I1
4:         return
5:     end if
6:     cur ← cur.right                       ▷ I3
7: end if
```

Before



After



Case 4. Insert 20, 40, 60, 80, then 35

Rules: I2 / I3, then I1

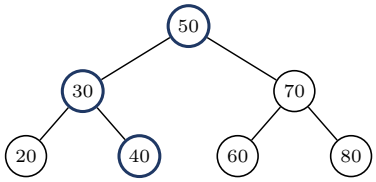
Key	Compared	Rules	Attached as
20	50, 30	I2, I2, I1	left of 30
40	50, 30	I2, I3, I1	right of 30
60	50, 70	I3, I2, I1	left of 70
80	50, 70	I3, I3, I1	right of 70
35	50, 30, 40	I2, I3, I2, I1	left of 40

35 goes left at 50 because $35 < 50$, and right at 30 because $35 > 30$. The side is the comparison with the current node. 3 comparisons.

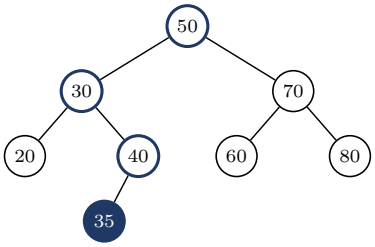
```
1: loop
2:     if key < cur.key then
3:         if cur.left = NULL then
```

```
4:              cur.left ← newNode; return                ▷ I1
5:          end if
6:          cur ← cur.left                                ▷ I2
7:      else if key > cur.key then
8:          if cur.right = NULL then
9:              cur.right ← newNode; return               ▷ I1
10:         end if
11:         cur ← cur.right                               ▷ I3
12:     end if
13: end loop
```

Before 35



After: 35 left of 40

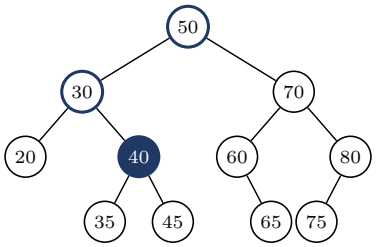


Case 5. Finish the tree, then insert 40 again — Rules: I4

45 compared with 50, 30, 40 (I2, I3, I3, I1): right of 40. 65 compared with 50, 70, 60 (I3, I2, I3, I1): right of 60. 75 compared with 50, 70, 80 (I3, I3, I2, I1): left of 80. Height 3. Inorder 20 30 35 40 45 50 60 65 70 75 80. Inserting 40 again walks 50, 30, 40 and hits I4, so the tree is unchanged. An equal key is neither smaller nor larger, so it has no child slot. Keys stay distinct.

```
1: if key = cur.key then
2:     return                                             ▷ I4
3: end if
```

Final tree. The second 40 stops on the marked path.



Case 6. Sorted keys 20, 30, 40, 50, 60 — Rules: I1, then I3 + I1

20 is the root (I1). Every later key is larger than every key already stored, so each step is I3 and the right child is NULL (I1). Height $n - 1 = 4$. The 11-key tree has height 3, so sorted input makes the taller tree.

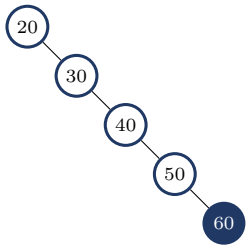
BUILD-FROM-ARRAY(data, n)

```
1: for i ← 0 to n − 1 do
2:     INSERT(data[i])
3: end for
```

Before

(empty)

Height 4



(g) All 11 insertions

Step	Insert	Compared	Rules	Parent	Side
1	50	—	I1	—	root
2	30	50	I2, I1	50	Left
3	70	50	I3, I1	50	Right
4	20	50, 30	I2, I2, I1	30	Left
5	40	50, 30	I2, I3, I1	30	Right
6	60	50, 70	I3, I2, I1	70	Left
7	80	50, 70	I3, I3, I1	70	Right
8	35	50, 30, 40	I2, I3, I2, I1	40	Left
9	45	50, 30, 40	I2, I3, I3, I1	40	Right
10	65	50, 70, 60	I3, I2, I3, I1	60	Right
11	75	50, 70, 80	I3, I3, I2, I1	80	Left

(h) INSERT, and the deepest insert

Rules: I1–I4

The deepest inserts are 35, 45, 65, and 75. Each compares with 3 nodes, equal to the height: the new leaf hangs at depth 3.

INSERT(key)

```
1: newNode ← a new node with key
2: if root = NULL then
3:     root ← newNode; return                                  ▷ I1
4: end if
5: cur ← root
6: loop
7:     if key < cur.key then
8:         if cur.left = NULL then
9:             cur.left ← newNode; return                      ▷ I1
10:        end if
11:        cur ← cur.left                                      ▷ I2
12:    else if key > cur.key then
13:        if cur.right = NULL then
14:            cur.right ← newNode; return                     ▷ I1
15:        end if
16:        cur ← cur.right                                     ▷ I3
17:    else
18:        return                                              ▷ I4
19:    end if
20: end loop
```

Problem 2. Search

One path from the root. The comparison count is the number of keys compared. Reaching NULL is S1 and does not add a comparison.

SEARCH(target) used by every case

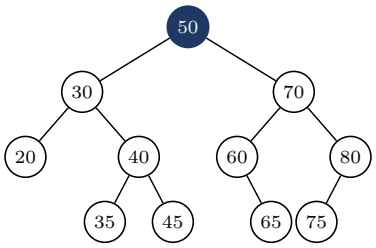
```
1: cur ← root
2: while cur ≠ NULL do
3:     if target = cur.key then
4:         return FOUND                                        ▷ S2
5:     else if target < cur.key then
6:         cur ← cur.left                                      ▷ S3
7:     else
8:         cur ← cur.right                                     ▷ S4
9:     end if
10: end while
11: return NOT FOUND                                           ▷ S1
```

Case 1. Search 50

Rules: S2

$50 = 50$ at the root. 1 comparison.

Path: 50

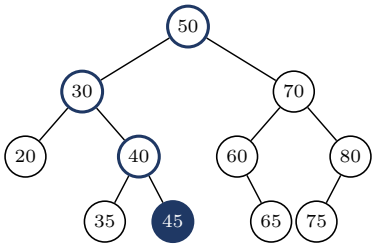


Case 2. Search 45

Rules: S3, S4, S4, S2

$45 < 50$ left, $45 > 30$ right, $45 > 40$ right, $45 = 45$. Path $50 \to 30 \to 40 \to 45$. 4 comparisons. The right subtree of 50 is skipped.

Path: $50 \to 30 \to 40 \to 45$

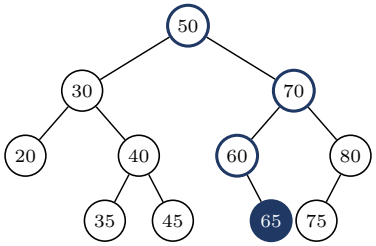


Case 3. Search 65

Rules: S4, S3, S4, S2

$65 > 50$ right, $65 < 70$ left, $65 > 60$ right, $65 = 65$. Path $50 \to 70 \to 60 \to 65$. 4 comparisons.

Path: $50 \to 70 \to 60 \to 65$

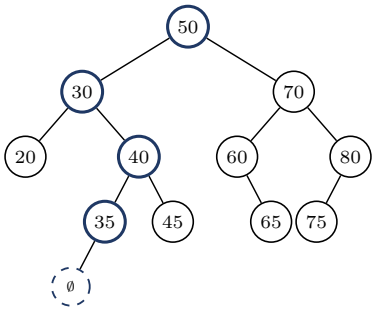


Case 4. Search 33

Rules: S3, S4, S3, S3, S1

$33 < 50$, $33 > 30$, $33 < 40$, $33 < 35$, then the left child of 35 is NULL. Path $50 \to 30 \to 40 \to 35 \to$ NULL. 4 comparisons. Inserting 33 would make it the left child of 35.

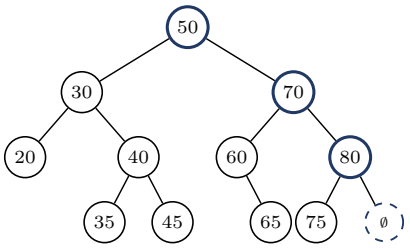
Stops on the missing left child of 35



Case 5. Search 90 — Rules: S4, S4, S4, S1

90 is greater than 50, 70, and 80, so the walk only goes right and falls off 80. Path $50 \to 70 \to 80 \to$ NULL. 3 comparisons. Inserted, 90 would be the right child of 80.

Stops on the missing right child of 80

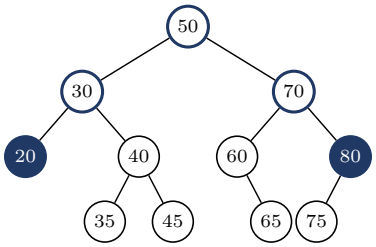


Case 6. Minimum and maximum — Rules: P2

No target. Minimum follows left children until the left pointer is NULL: $50 \to 30 \to 20$. Maximum follows right children: $50 \to 70 \to 80$.

```
1: while node.left ≠ NULL do
2:     node ← node.left
3: end while
4: return node.key                                   ▷ FIND-MIN: 20
5: while node.right ≠ NULL do
6:     node ← node.right
7: end while
8: return node.key                                   ▷ FIND-MAX: 80, called from the root
```

Left spine is the minimum. Right spine is the maximum.



Search table

Case	Target	Nodes compared	Rules	Result	Comparisons
1	50	50	S2	found	1
2	45	50, 30, 40, 45	S3, S4, S4, S2	found	4
3	65	50, 70, 60, 65	S4, S3, S4, S2	found	4
4	33	50, 30, 40, 35	S3, S4, S3, S3, S1	not found	4
5	90	50, 70, 80	S4, S4, S4, S1	not found	3
6	min / max	50, 30, 20 / 50, 70, 80	P2	20 / 80	0

(g) Longest search: 4 comparisons $= h + 1$. A path has at most one node per level, and this tree has height 3.

Problem 3. Delete

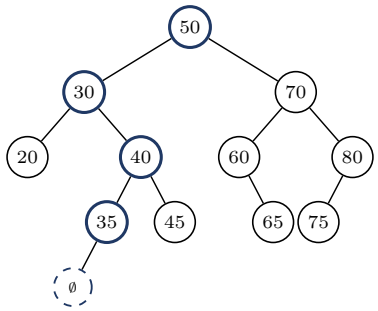
Each case starts again from the 11-key tree. Search first, then use the number of children.

Case 1. Delete 33 — Rules: S1

Same failed search as above: $50 \to 30 \to 40 \to 35 \to \text{NULL}$. Nothing is removed.

```
1: if node = NULL then
2:     return NULL                                   ▷ S1
3: end if
```

Unchanged

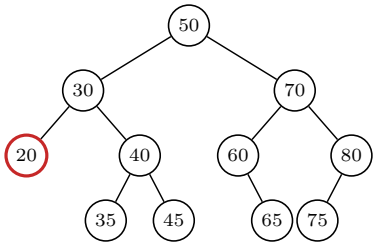


Case 2. Delete 20 — Rules: D1

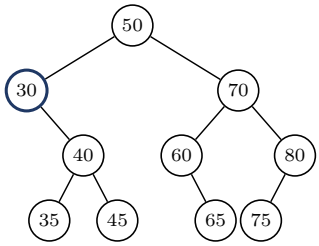
20 is a leaf. The left pointer of 30 becomes NULL.

```
1: if node.left = NULL and node.right = NULL then
2:     return NULL                                   ▷ D1
3: end if
```

Before



After

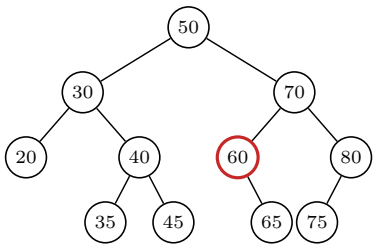


Case 3. Delete 60 — Rules: D2

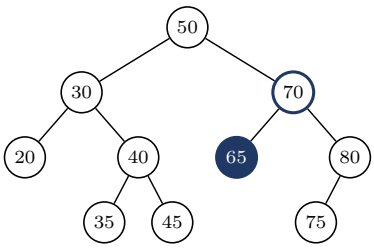
60 has only a right child, 65. `70.left` now points at 65.

```
1: if node.left = NULL then
2:     return node.right                             ▷ D2
3: end if
```

Before



After

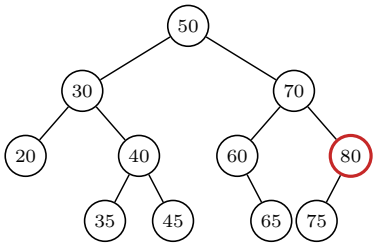


Case 4. Delete 80 — Rules: D2

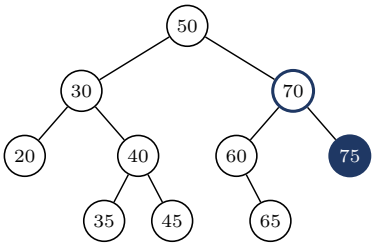
80 has only a left child, 75. `70.right` now points at 75.

```
1: if node.right = NULL then
2:     return node.left                                  ▷ D2
3: end if
```

Before



After

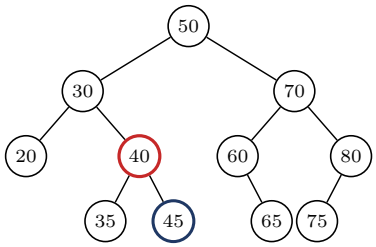


Case 5. Delete 40 — Rules: D3, then D1

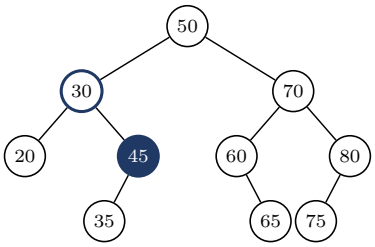
40 has two children, so D3. Successor: go right once. 45 has no left child, so the successor is 40's own right child. Copy 45 into the node, then delete 45. That node is a leaf, so the second step is D1.

```
1: s ← node.right                                       ▷ 45, and s.left is NULL
2: node.key ← s.key
3: node.right ← DELETE(node.right, s.key)               ▷ D1
```

Before



After

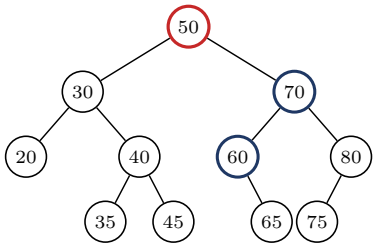


Case 6. Delete 50 — Rules: D3, then D2

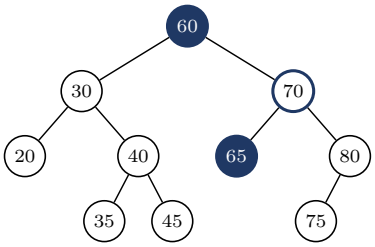
50 has two children. Successor: right to 70, then left to 60. Copy 60 into the root. The old 60 has a right child, 65, and no left child, so deleting it is D2: `70.left` points at 65. A successor has no left child, so removing it is D1 or D2.

```
1: s ← node.right                                       ▷ 70
2: while s.left ≠ NULL do
3:     s ← s.left                                       ▷ 60
4: end while
5: node.key ← s.key
6: node.right ← DELETE(node.right, s.key)               ▷ D2, returns 65
```

Before



After



Delete table

Case	Delete	Children	Successor	Rules	Level order after
1	33	not found	—	S1	50 30 70 20 40 60 80 35 45 65 75
2	20	none	—	D1	50 30 70 40 60 80 35 45 65 75
3	60	right: 65	—	D2	50 30 70 20 40 65 80 35 45 75
4	80	left: 75	—	D2	50 30 70 20 40 60 75 35 45 65
5	40	35 and 45	45	D3, D1	50 30 70 20 45 60 80 35 65 75
6	50	30 and 70	60	D3, D2	60 30 70 20 40 65 80 35 45 75

(g) DELETE

Rules: S1, D1, D2, D3

DELETE(node, key) returns the new subtree root

```
1: if node = NULL then
2:     return NULL                                          ▷ S1
3: end if
4: if key < node.key then
5:     node.left ← DELETE(node.left, key)
6: else if key > node.key then
7:     node.right ← DELETE(node.right, key)
8: else
9:     if node.left = NULL then
10:        return node.right                                ▷ D1, or D2
11:    end if
12:    if node.right = NULL then
13:        return node.left                                 ▷ D2
14:    end if
15:    s ← node.right
16:    while s.left ≠ NULL do
17:        s ← s.left                                       ▷ successor
18:    end while
19:    node.key ← s.key
20:    node.right ← DELETE(node.right, s.key)
21: end if
22: return node
```

(h) Why the inorder successor

The successor is the next key in sorted order: the smallest key in the right subtree. It is larger than everything on the left and smaller than everything else on the right, so copying it keeps the BST property. It has no left child, so removing it is D1 or D2. The deepest node can be any key, and putting it in the hole can break that order.